

使用说明

本文需要读者对 C 语言有一定的了解作为基础

本文中所涉及的例子可以用本站提供的‘全自动化 Makefile’一文中提供的 Makefile 进行编译

读者如果在 Linux 下，可以直接使用，Windows 用户需要 Cygwin 环境

本文中的工具，需要用户安装 flex 和 bison 软件包

1. 介绍

编译器是软件开发中的核心部件，其作用是其他任何软件所不能取代的。编译器在工作过程中，往往完成如下的任务：

1. 读取源代码并且获得程序的结构描述
2. 分析程序结构，并且生成相应的目标代码

在 UNIX 早期时代，编写一个编译器是一件非常耗时的工作。人们为了简化开发过程，开发了 Lex 和 YACC 程序来解决第一个任务，根据用户描述的语言，生成能够解决问题的 C/C++ 语言代码，供开发者使用。

1. 将源代码文件分解为各种词汇（Lex）
2. 找到这些词汇的组成方式（YACC）

GNU 软件协会开发了 Flex 和 BISON，其功能与 LEX 和 YACC 基本兼容，并且在 Lex 和 YACC 提供的功能的基础上进行了各种扩展。

入门

Lex 能够用来编写那些输入数据流（字符串）能够用正则表达式描述的程序，它可以根据正则表达式的描述，将输入数据流分类为各类词汇，为后来的语法分析做准备。

. 正则表达式

正则表达式是通过对各种词组类型所包含的字符类型的归纳，描述所需词组组成格式的方法，比如下面的例子描述了所有数字类型的字符串，表明无论在何时起，只要有 0-9 字符出现，就进入该状态，说明是字符，直到非 0-9 字符结束：

[09]+

为了简化书写起见，也可以写成如下的格式：

[0-9]+

对于任意单词，其中只能包含字母，所以单词的正则表达式为：

[a-zA-Z]+

Flex 支持如下类型的正则表达式:

x	符合字符串"x"
.	除了换行以外的任何字符
[xyz]	一个字符类, 在这个例子中, 输入必须符合要么是'x' 要么是'y' 要么是'z'
[abj-oZ]	一个带范围的字符类, 符合任何满足'a', 'b', 从'j' 到'o' 还有'Z'
[^A-Z]	一个取反的字符类, 比如任何字母除了大写字母。
[^A-Z\n]	任何字符除了大写字母和换行
r*	零个或更多 r, r 可以是任意正则表达式
r+	一个或更多 r
r?	零个或最多一个 r
r{2,5}	任何 2 到 5 个 r
r{2,}	2 个或更多 r
r{4}	正好 4 个 r
{name}	对 name 的扩展
"[xyz]\\"foo"	符合正则表达式 [xyz]"foo 的字符串
\X	如果 X 是一个'a', 'b', 'f', 'n', 'r', 't', 或者'v', 则按照 ASCII 码\x 转义符进行处理, 否则, 其他的'X' 将用来做取消处理符
处理	
\0	一个 ASCII 码空字符
\123	内容为八进制 123 的 char 型
\x2a	内容为十六进制 0x2a 的 char 型
(r)	符合一个 r, 括号是用来越过优先级的
rs	正则表达式 r, 紧跟着一个
r s	要么是 r 要么是 s
^r	一个 r, 但是必须是在一行的开始
r\$	一个 r, 但是必须是在行末
<s>r	一个 r, 但是之前字符串必须符合条件 s
<s1, s2, s3>r	同上, 但是必须之前字符串符合 s1 或者 s2 或者 s3
<*>r	不考虑开始字符串类型, 只符合 r
<<EOF>>	文件末尾
<s1, s2><<EOF>>	前面符合 s1 或者 s2 的文件末尾

. 第一个 Lex 代码

按照上一节我们讲述的正则表达式例子, 我们尝试第一次使用 Lex 来产生我们所需要的程序, 实践一遍 Lex 的使用以及 gcc 编译器如何编译和生成所需的二进制。Flex 甚至 Bison 代码都有如下的编写格式:

```
/* 定义段 */  
%%  
/* Flex、Bison 代码段 (规则) */
```



```

\n                printf("\n");                /* 换行 */
[ \t]+            /* 忽略空白字符 */
%%
int yywrap(void)    /* 当词法分析器到了文件末尾做什么 */
{
    return 1;        /* 返回 1，说明停止前进，0 则继续 */
}

void yyerror(char *s) /* 错误信息打印函数 */
{
    fprintf(stderr, "%s\n", s);
    return 0;
}

int main(int argc, char *argv[])
{
    FILE *fp;
    fp = fopen(argv[1], "r"); /* 首先打开要被处理的文件（参数 1） */

    yyin = fp;                /* yyin 是 lex 默认的文件输入指针，则不处
理控制台输入 */

    yylex();                  /* 调用 lex 的工作函数，从这里开始，lex
的词法分析器开始工作 */

    fclose(fp);
    return 0;
}

```

到这里，我们已经完全可以对一个包含各种类型词组的源代码文件进行分析，得出其中各类型词组的排列顺序。在一个规范的基于语法的源代码中，词组的顺序从一定意义上来说，就是语法。对于源代码，lex 的处理能力也就到这里为止，但是我们并没有完全展示 lex 的所有功能，读者如果有兴趣，可以继续深入阅读本文提供的参考文献。如何进行语法分析？我们在下面的章节中开始讲述 Bison 的基本使用方法，以及如何使用 Bison 进行语法分析。

入门

. 基础理论

Bison 采用与 YACC 相同的描述语言，这种语言是 BNF 语法 (Backus Naur Form) 的一种，最早被 John Backus 和 Peter Naur 用于 ALGOL60 语言的开发工作。BNF 语法可以用来表达与内容无关的，基于语法的语言，几乎所有的现代编程语言都可以用 BNF 进行描述。作为一个例子，一个进行加法和乘法的数学表达式语法可以如下表达：

```

E : E '+' E
E : E '*' E
E : id

```

在这三句中，E 在 Bison 语言中代表表达式，一个表达式可以是一个数字，也可以是一个变量名称，也可以是几个变量名称的组合，比如：

```

1
1+2
a
a+b*c

```

而以上三句 Bison 语言就能够表达这些语法结构。

初探

我们在下面作一个计算器，通过这个实例让大家明白 Flex 和 Bison 的相互关系和如何共同工作。首先，建立文件，并输入以下内容：

```

/* 计算器的词法分析器描述，Flex 语言 */
%{
/* 直接翻译为 C 语言 */
#include <> /* 包含标准库文件 */
void yyerror(char *); /* 这是我们上面用到的错误报告函数 */
#include "" /* 这个头文件由 Bison 自动生成 */
}%
%%
[0-9]+ {
    yylval = atoi(yytext); /* yytext 是 flex 内部用于指向当前词汇
的字符串指针 */
    return INTEGER; /* INTEGER 是从中包含过来的，在 Bison
中定义 */
}
[+\n] return *yytext;
[ \t] ; /* 跳过空白字符 */
. yyerror("invalid character"); /* 产生一个错误，说明是无效字符 */
%%
int yywrap(void)
{
    return 1; /* 文件结束时退出 */
}

```

以上就是计算器使用的 Flex 语言，描述了我们将会遇到的各种词汇的格式，比如[0-9]+说明了，只有连续的从'0'到'9'的字符串，才被分析为 INTEGER 类型，如果遇到制表符、空格等，直接忽略，遇到加减符号则返回字符指针，其它的则报告语法错误。下面是我们的 Bison 语法文件：

```

/* 注：您先抄写，注解见下文 */
%{
#include <>
int yylex(void);

```

```

void yyerror(char *);
%}
%token INTEGER                                /* Flex 语言中 INTEGER 定义在此 */
%%
program:
    program expr '\n'          { printf("%d\n", $2); }
    |
    ;
expr:
    INTEGER                    { $$ = $1; }
    | expr '+' expr            { $$ = $1 + $3; } /* (1) */
    | expr '-' expr            { $$ = $1 - $3; } /* (2) */
    ;
%%
void yyerror(char *s)
{
    fprintf(stderr, "%s\n", s);
}
int main(void)
{
    yyparse();
    return 0;
}

```

编译过程:

```

flex
bison -d          # 注意-d, 用于产生对应的头文件
gcc -o test2

```

在这个例子中, 我们遇到了许多没有见过的用法, Bison 的书写格式基本与 Flex 的相同, 只是规则的定义语法不同。其中, $\$N$ (N 为数字) 代表语法描述中, 第 N 个词汇的内容, 比如(1)中的 $\$1$ 代表 '+' 左边的 `expr`, $\$3$ 代表右边 `expr` 的内容, 其中的 N 是指当前语法的词汇顺序, 从 1 开始计数。而 $$$$ 则代表这一段语法处理完后的结果, 每一个语法对应的处理都有输入 $\$N$ 和输出 $$$$ 。该例子中还有一个特殊的地方, 就是第归调用, 在 Bison 中, 语法规则可以是第归的, 这也是 Bison 之处 (或者说是 YACC 的强大之处)。举个例子:

```

program:
    program expr '\n'          { printf("%d\n", $2); }
    |
    ;

```

这里, `program` 的定义就是任何符合 `program` 的表达式后面紧接着 `expr` 和 '\n', 扫描到该表达式后, 将 `expr` 处理的结果打印到屏幕。最后的 | 是“或者”的意思, 也就是说 `program` 也可以是空的, 什么都不写, 分号代表该语义定义结束。有了第归之后, Bison 才可以说是一个能够应对任何状况的语法分析器, 当然, 这里还需要读者对以上所提供代码进行深入的研究和分析, 考虑清楚后, 会发现无论是 C 语言, 还是 Bison, 第归永远是一个神奇的解决方案。

. 计算器程序的深入研究

以上我们设计的计算器只能进行加减计算，并且里面还有一些软件缺陷，对于一个实用的计算器来说，我们必须能够支持：

1. 变量定义
2. 变量赋值
3. 变量与数字立即处理

于是我们假想设计出来的计算器能有如下的操作过程（输入和输出）：

输入：3 * (4 + 5)

结果：27

输入：x = 3 * (4 + 5)

输入：y = 5

输入：x

结果：27

输入：y

结果：5

输入：x + 2 * y

结果：37

这样看，这个计算器的功能已经相当强大了，下面我们着手实现，首先修改上面例子的 Flex 文件为如下：

```
%{
    #include <>
    void yyerror(char *);
    #include ""
}%
%%
[a-z] { /* 变量类型词汇，限制：变
量只能用一个字符 */
        yylval = *yytext - 'a';
        return VARIABLE;
}
[0-9]+ { /* 整数 */
        yylval = atoi(yytext);
        return INTEGER;
}
[+-()=/*\n] { return *yytext; } /* 数学计算符号 */
[ \t] ; /* 跳过空白符号 */
%%
int yywrap(void)
{
    return 1;
}
```

下面是我们新的 Bison 文件：

```

%token INTEGER VARIABLE
%left '+' '-'
%left '*' '/'
%{
    void yyerror(char *);
    int yylex(void);
    int sym[26];
}%
%%
program:
    program statement '\n'
    |
    ;
statement:
    expr                                     { printf("%d\n", $1); }
    | VARIABLE '=' expr                   { sym[$1] = $3; }
    ;
expr:
    INTEGER
    | VARIABLE                           { $$ = sym[$1]; }
    | expr '+' expr                      { $$ = $1 + $3; }
    | expr '-' expr                      { $$ = $1 - $3; }
    | expr '*' expr                      { $$ = $1 * $3; }
    | expr '/' expr                      { $$ = $1 / $3; }
    | '(' expr ')'                       { $$ = $2; }
    ;
%%
void yyerror(char *s)
{
    fprintf(stderr, "%s\n", s);
    return 0;
}
int main(void)
{
    yyparse();
    return 0;
}

```

现在我们对该例子中引入的新功能介绍，%left, %right, %token 都是用来声明词汇的，区别在于，%token 声明的词汇与左右优先级无关，而%left 的处理时，先处理左边的，%right 先处理右边的，例如遇到字符串“1 - 2 - 5”，到底是处理为“(1 - 2) - 5”，还是处理为“1 - (2 - 5)”？%left 处理为前者，而%right 处理为后者（注：第一个计算器代码中就有这个缺陷，比如执行 1-2+3，得到的结果就是-4，作为一个练习，读者可以使用这里讲解的%left 自行更正）。

和 Bison 高级应用

在下面的例子中，我们便写了一个更高级的计算器程序，其操作实例如下：

```
x = 0;
while(x < 3) {
    print(x);
    x = x + 1;
}
```

例子中得到的输出结果如下：

```
0
1
2
```

首先是我们的全局头文件：

```
typedef enum {
    typeCon,
    typeId,
    typeOpr
} nodeEnum;

/* constants */
typedef struct {
    int value;          /* value of constant */
} conNodeType;

/* identifiers */
typedef struct {
    int i;              /* subscript to sym array */
} idNodeType;

/* operators */
typedef struct {
    int oper;           /* operator */
    int nops;           /* number of operands */
    struct nodeTypeTag *op[1]; /* operands (expandable) */
} oprNodeType;

typedef struct nodeTypeTag {
    nodeEnum type;      /* type of node */

    /* union must be last entry in nodeType */
    /* because oprNodeType may dynamically increase */
    union {
        conNodeType con;          /* constants */
    };
};
```

```

        idNodeType id;                /* identifiers */
        oprNodeType opr;              /* operators */
    };
} nodeType;

extern int sym[26];
下面是 Flex 语言文件:
%{
#include <>
#include ""
#include ""
void yyerror(char *);
}%

%%

[a-z]          {
                = *yytext - 'a';
                return VARIABLE;
            }

[0-9]+         {
                = atoi(yytext);
                return INTEGER;
            }

[-()<>=+*/;{}.] {
                return *yytext;
            }

">="          return GE;
"<="          return LE;
"=="          return EQ;
"!="          return NE;
"while"        return WHILE;
"if"           return IF;
"else"         return ELSE;
"print"        return PRINT;

[ \t\n]+      ;                /* ignore whitespace */
.              yyerror("Unknown character");

%%

```

```
int yywrap(void)
{
    return 1;
}
```

然后是我们的 Bison 文件:

```
%{
#include <>
#include <>
#include <>
#include ""

/* prototypes */
nodeType *opr(int oper, int nops, ...);
nodeType *id(int i);
nodeType *con(int value);
void freeNode(nodeType *p);
int yylex(void);

void yyerror(char *s);
int sym[26];
%}

%union {
    int iValue;          /* integer value */
    char sIndex;         /* symbol table index */
    nodeType *nPtr;      /* node pointer */
};

%token <iValue> INTEGER
%token <sIndex> VARIABLE
%token WHILE IF PRINT
%nonassoc IFX
%nonassoc ELSE

%left GE LE EQ NE '>' '<'
%left '+' '-'
%left '*' '/'
%nonassoc UMINUS

%type <nPtr> stmt expr stmt_list

%%

program:
```

```

function                                { exit(0); }
;

function:
function stmt                          { ex($2); freeNode($2); }
|
;

stmt:
';'                                    { $$ = opr(';', 2, NULL,
NULL); }
| expr ';'                            { $$ = $1; }
| PRINT expr ';'                      { $$ = opr(PRINT, 1, $2); }
| VARIABLE '=' expr ';'              { $$ = opr('=', 2, id($1), $3); }
| WHILE '(' expr ')' stmt            { $$ = opr(WHILE, 2, $3, $5); }
| IF '(' expr ')' stmt %prec IFX{ $$ = opr(IF, 2, $3, $5); }
| IF '(' expr ')' stmt ELSE stmt{ $$ = opr(IF, 3, $3, $5, $7); }
| '{' stmt_list '}'                  { $$ = $2; }
;

stmt_list:
stmt                                  { $$ = $1; }
| stmt_list stmt                      { $$ = opr(';', 2, $1, $2); }
;

expr:
INTEGER                              { $$ = con($1); }
| VARIABLE                           { $$ = id($1); }
| '-' expr %prec UMINUS              { $$ = opr(UMINUS, 1, $2); }
| expr '+' expr                      { $$ = opr('+', 2, $1, $3); }
| expr '-' expr                      { $$ = opr('-', 2, $1, $3); }
| expr '*' expr                      { $$ = opr('*', 2, $1, $3); }
| expr '/' expr                      { $$ = opr('/', 2, $1, $3); }
| expr '<' expr                       { $$ = opr('<', 2, $1, $3); }
| expr '>' expr                       { $$ = opr('>', 2, $1, $3); }
| expr GE expr                      { $$ = opr(GE, 2, $1, $3); }
| expr LE expr                      { $$ = opr(LE, 2, $1, $3); }
| expr NE expr                      { $$ = opr(NE, 2, $1, $3); }
| expr EQ expr                      { $$ = opr(EQ, 2, $1, $3); }
| '(' expr ')'                      { $$ = $2; }
;

```

%%

```

#define SIZEOF_NODETYPE ((char*)&p->con - (char*)p)

nodeType *con(int value) {
    nodeType *p;
    size_t nodeSize;

    /* allocate node */
    nodeSize = SIZEOF_NODETYPE + sizeof(conNodeType);
    if((p = malloc(nodeSize)) == NULL)
        yyerror("out of memory");

    /* copy information */
    p->type = typeCon;
    p-> = value;

    return p;
}

nodeType *id(int i) {
    nodeType *p;
    size_t nodeSize;

    /* allocate node */
    nodeSize = SIZEOF_NODETYPE + sizeof(idNodeType);
    if((p = malloc(nodeSize)) == NULL)
        yyerror("out of memory");

    /* copy information */
    p->type = typeId;
    p-> = i;

    return p;
}

nodeType *opr(int oper, int nops, ...) {
    va_list ap;
    nodeType *p;
    size_t nodeSize;
    int i;

    /* allocate node */
    nodeSize = SIZEOF_NODETYPE + sizeof(oprNodeType) +
        (nops - 1) * sizeof(nodeType*);
    if((p = malloc(nodeSize)) == NULL)

```

```

        yyerror("out of memory");

    /* copy information */
    p->type = type0pr;
    p-> = oper;
    p-> = nops;
    va_start(ap, nops);
    for(i = 0; i < nops; i++)
        p->[i] = va_arg(ap, nodeType*);
    va_end(ap);
    return p;
}

```

```

void freeNode(nodeType *p) {
    int i;

    if(!p) return;
    if(p->type == type0pr) {
        for(i=0; i<p->; i++)
            freeNode(p->[i]);
    }
    free(p);
}

```

```

void yyerror(char *s) {
    fprintf(stdout, "%s\n", s);
}

```

```

int main(void) {
    yyparse();
    return 0;
}

```

上面的 Flex 和 Bison 代码所作的工作是根据语法建立语意描述树结构。延续我们上面计算器的例子，我们写出如何翻译这些语言的实现部分（对生成的树进行第归分析）：

```

#include <>
#include ""
#include ""

```

```

int ex(nodeType *p) {
    if(!p) return 0;
    switch(p->type) {
    case typeCon:
        return p->;
    case typeId:

```

```

        return sym[p->];
case type0pr:
    switch(p-> {
    case WHILE:
        while(ex(p->[0]))
            ex(p->[1]);
        return 0;
    case IF:
        if(ex(p->[0]))
            ex(p->[1]);
        else if(p-> > 2)
            ex(p->[2]);
        return 0;
    case PRINT:
        printf("%d\n", ex(p->[0]));
        return 0;
    case ';' :
        ex(p->[0]);
        return ex(p->[1]);
    case '=' :
        return sym[p->[0]->] = ex(p->[1]);
    case UMINUS:
        return -ex(p->[0]);
    case '+' :
        return ex(p->[0]) + ex(p->[1]);
    case '-' :
        return ex(p->[0]) - ex(p->[1]);
    case '*' :
        return ex(p->[0]) * ex(p->[1]);
    case '/' :
        return ex(p->[0]) / ex(p->[1]);
    case '<' :
        return ex(p->[0]) < ex(p->[1]);
    case '>' :
        return ex(p->[0]) > ex(p->[1]);
    case GE:
        return ex(p->[0]) >= ex(p->[1]);
    case LE:
        return ex(p->[0]) <= ex(p->[1]);
    case NE:
        return ex(p->[0]) != ex(p->[1]);
    case EQ:
        return ex(p->[0]) == ex(p->[1]);

```

```

    }
}

```

一般实际的编译器都是以汇编代码输出的，所以我们在这里进行一些深入研究，得出了另一个版本的 ex 函数实现，能够实现汇编代码的输出（）：

```

#include <>
#include ""
#include ""

static int lbl;

int ex(nodeType *p) {
    int lbl1, lbl2;

    if(!p) return 0;
    switch(p->type) {
    case typeCon:
        printf("\tpush\t%d\n", p->);
        break;

    case typeId:
        printf("\tpush\t%c\n", p-> + 'a');
        break;

    case typeOpr:
        switch(p-> {
        case WHILE:
            printf("L%03d:\n", lbl1 = lbl++);
            ex(p->[0]);
            printf("\tjs\tL%03d\n", lbl2 = lbl++);
            ex(p->[1]);
            printf("\tjz\tL%03d\n", lbl1);
            printf("L%03d:\n", lbl2);
            break;

        case IF:
            ex(p->[0]);
            if(p-> > 2) {
                /* if else */
                printf("\tjs\tL%03d\n", lbl1 = lbl++);
                ex(p->[1]);
                printf("\tjmp\tL%03d\n", lbl2 = lbl++);
                printf("L%03d:\n", lbl1);
                ex(p->[2]);
            }
        }
    }
}

```

```

        printf("L%03d:\n", lbl2);
    } else {
        /* if */
        printf("\tjs\tL%03d\n", lbl1 = lbl++);
        ex(p->[1]);
        printf("L%03d:\n", lbl1);
    }
    break;

case PRINT:
    ex(p->[0]);
    printf("\tprint\n");
    break;

case '=':
    ex(p->[1]);
    printf("\tpop\t%c\n", p->[0]-> + 'a');
    break;

case UMINUS:
    ex(p->[0]);
    printf("\tneg\n");
    break;

default:
    ex(p->[0]);
    ex(p->[1]);
    switch(p-> {
        case '+':      printf("\tadd\n"); break;
        case '-':      printf("\tsub\n"); break;
        case '*':      printf("\tmul\n"); break;
        case '/':      printf("\tdiv\n"); break;
        case '<':      printf("\tcompLT\n"); break;
        case '>':      printf("\tcompGT\n"); break;
        case GE:       printf("\tcompGE\n"); break;
        case LE:       printf("\tcompLE\n"); break;
        case NE:       printf("\tcompNE\n"); break;
        case EQ:       printf("\tcompEQ\n"); break;
    }

}

return 0;
}

```

使用方法：取消的编译，取而代之用即可。关于这个计算器的代码分析，请等待后续，（未完待续）

参考文献

1. Flex 手册（见 man flex）
2. Bison 手册（见 man bison）
3. bert hubert, Lex and YACC promoter/HOWTO, [lex-yacc/](#)
4. Tom Niemann, A Compact Guide To LEX & YACC,
- 5.